Git & GitHub

Version Control

Version Control(버전 관리)

: 소프트웨어의 개발 과정에서 파일의 변경 이력을 기록하고 관리하는 시스템

- 소스코드, 문서, 설계 자료 등 다양한 파일의 변경 사항을 추적하고 이전 버전으로 복구할 수 있도록 관리

- 여러 개발자가 동시에 하나의 프로젝트를 협업할 수 있도록 돕는 도구

Version Control의 필요성

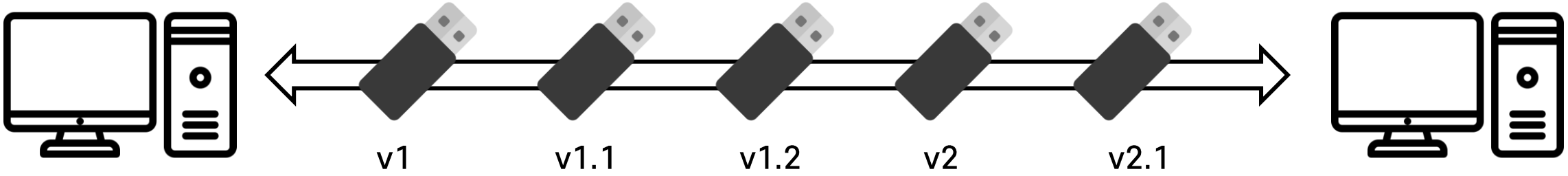
- 여러 개발자가 동시에 같은 프로젝트를 개발
- 코드 수정 이력 관리
- 이전 버전으로 복구 필요
- 실험적인 기능 개발 필요
- 코드 백업 필요

기능 →

- 코드 버전 관리
- 변경 이력 기록(누가 언제 무엇을 수정)
- 백업 및 복구
- 협업 개발 지원
- branch-off 와 merge

Version Control

Version Control 이전의 옛날 방식

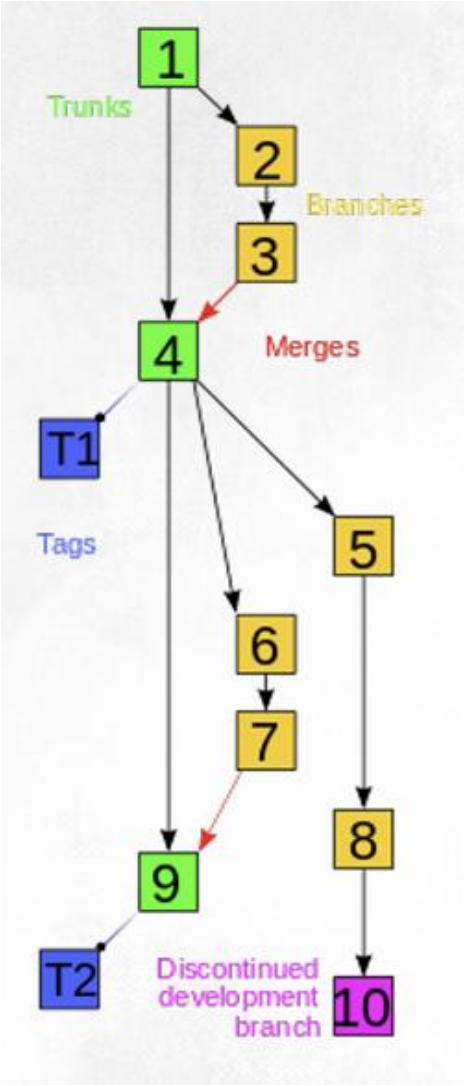


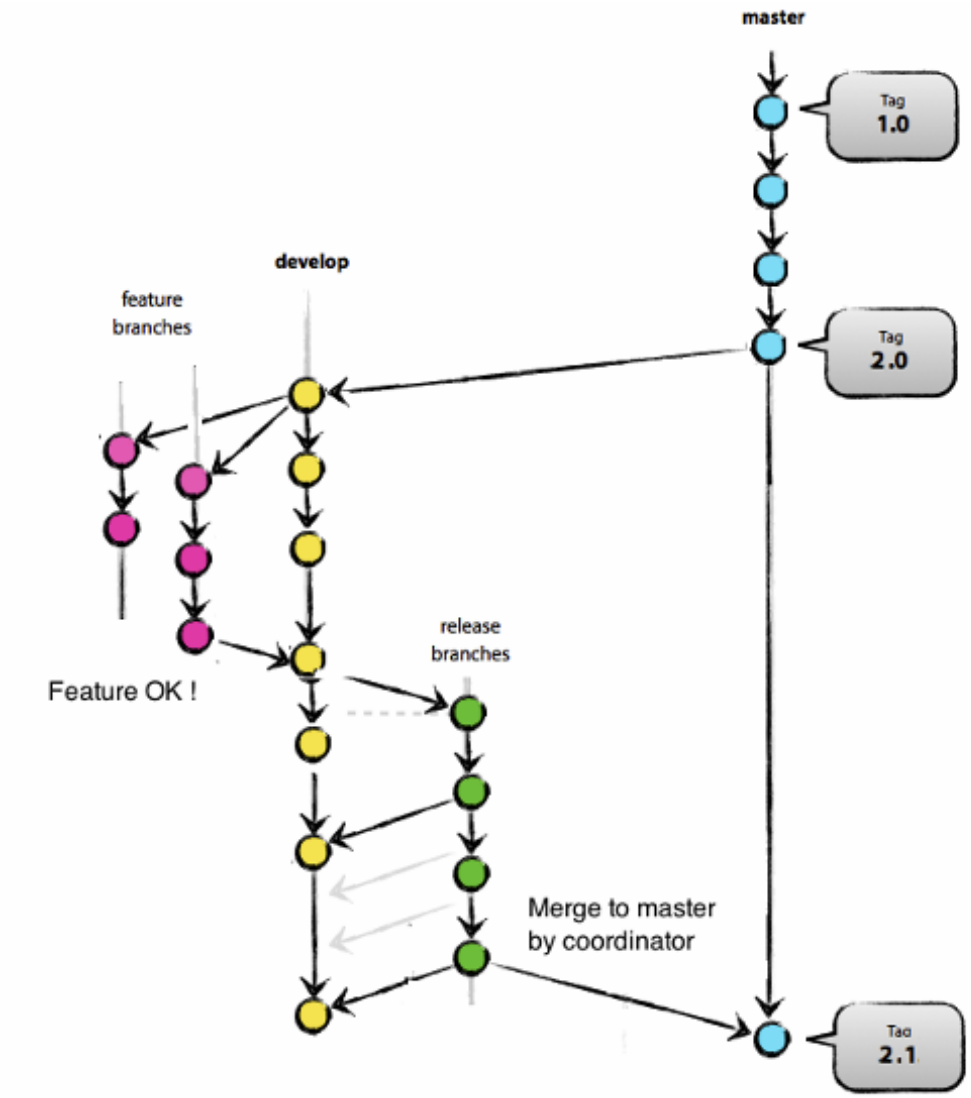
- USB로 코드 전달, 이메일로 파일 공유, 파일 이름에 버전 붙이기…

옛날 방식 문제점

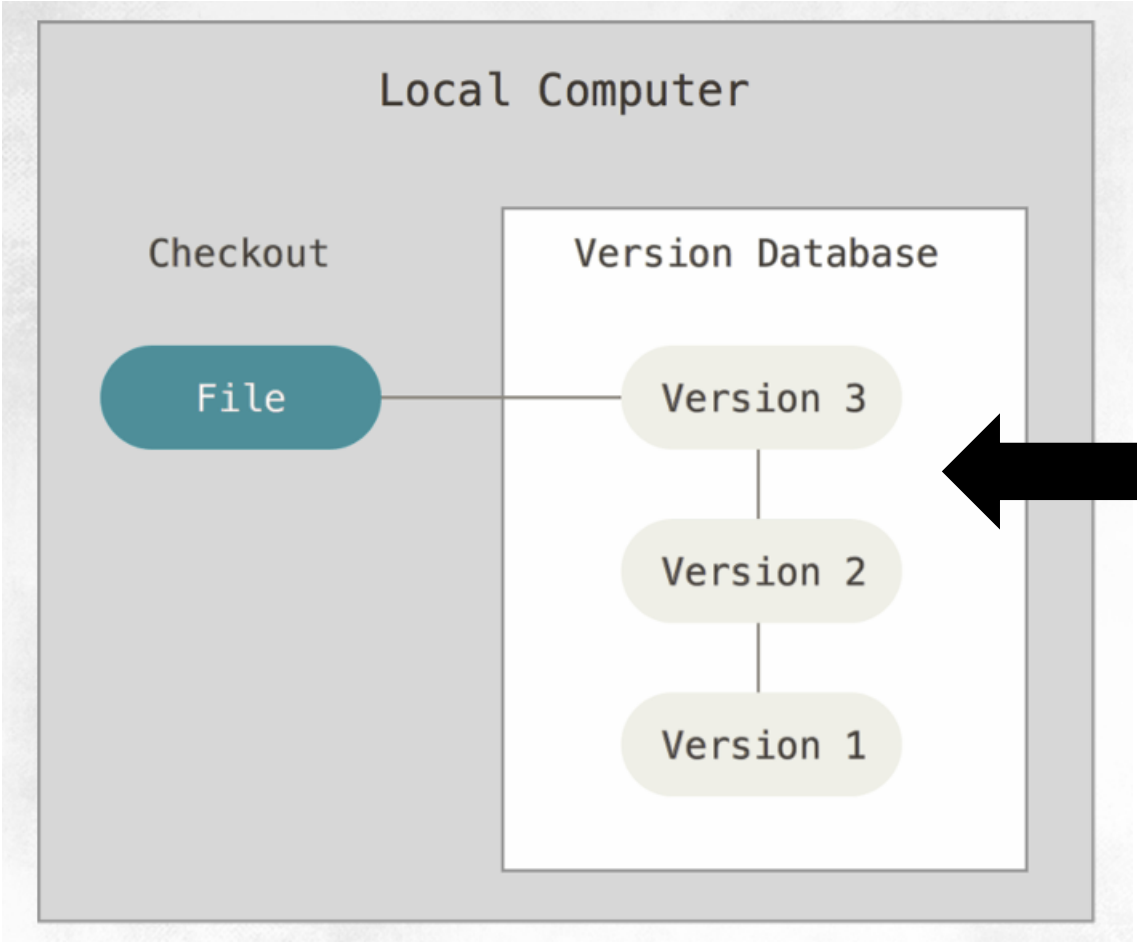
- 이전 버전 찾기 어려움
- 협업이 매우 비효율적
- 누가 수정했는지 알기 어려움
- 백업 관리 어려움

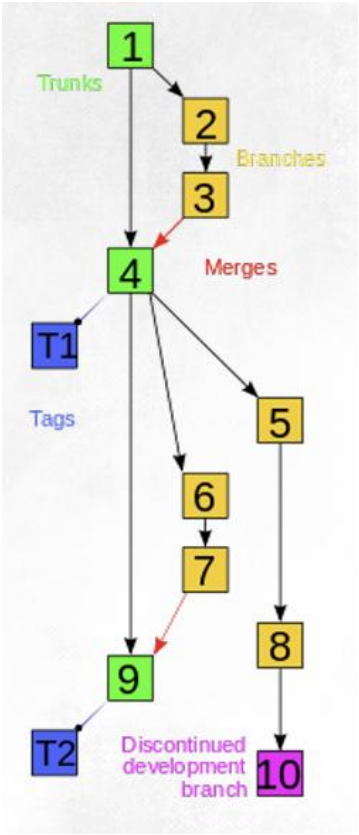
소프트웨어 개발 흐름


1
Trunks
2
Branches
3
4
Merges
T1
Tags
5
6
7
8
9
T2
Discontinued
development
branch
10

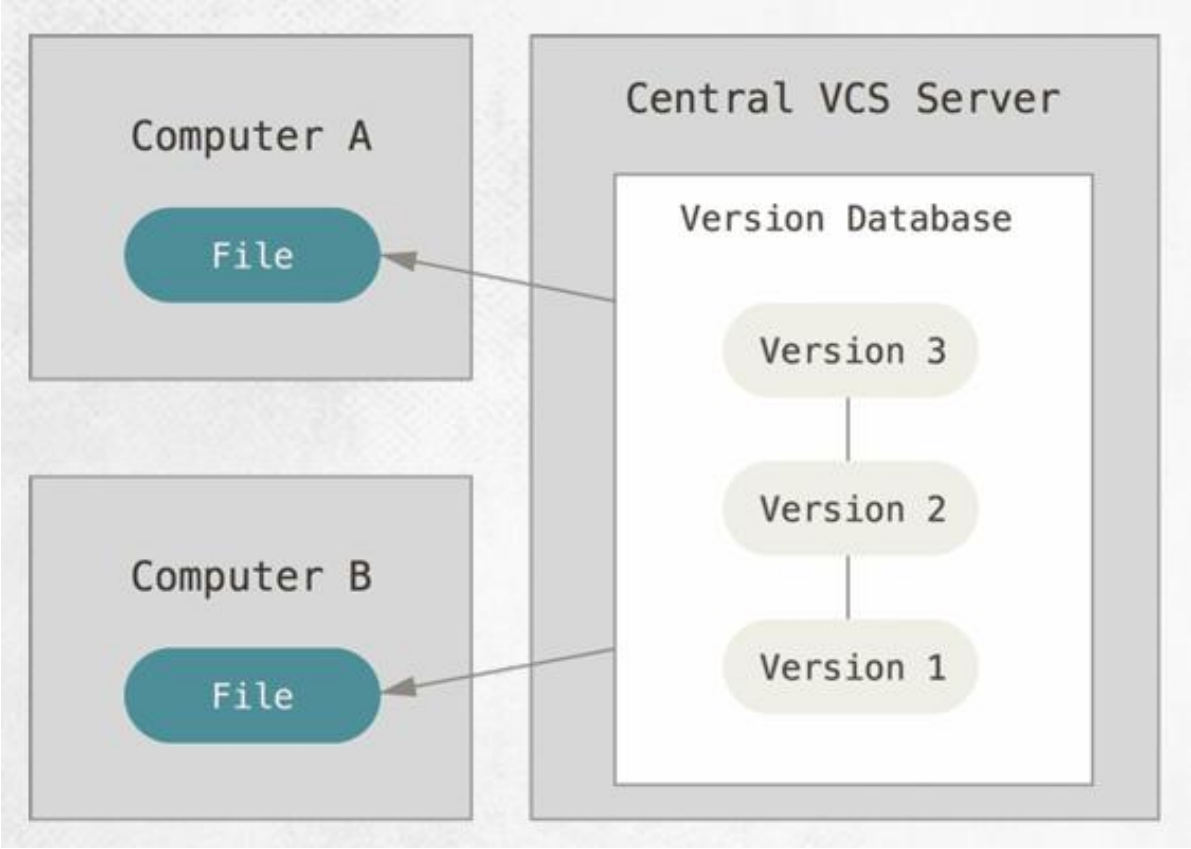

master
Tag
1.0
develop
feature
branches
Tag
2.0
release
branches
Feature OK !
Merge to master
by coordinator
Tag
2.1

Version Control on Single Machine





Centralized Version Control System



구조: server-client 구조

중앙 서버가 하나 존재하고 개발자들은 서버에 접속해 코드를 로컬환경으로 가져와 작업한다.

특징:

중앙 서버가 모든 데이터를 관리

개발자는 서버에서 코드 다운로드 후 작업

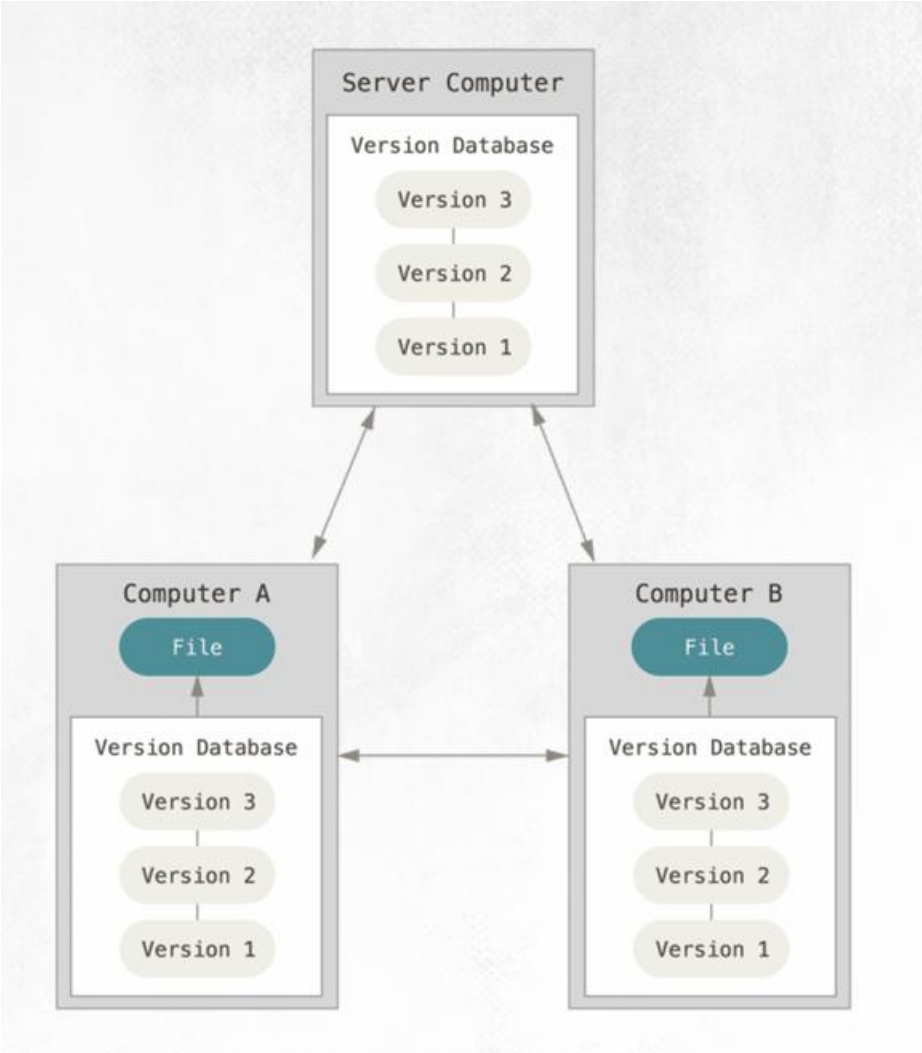
대표적인 시스템:

CVS, SVN(Apache Subversion)

단점:

네트워크 의존
서버 다운 시 작업 불가능

Distributed Version Control System



구조:

합의된 main server
각 개발자가 전체 저장소를 로컬 컴퓨터에 복사해 작업

특징:

모든 개발자가 전체 저장소를 로컬에 보유

네트워크 없이도 작업 가능

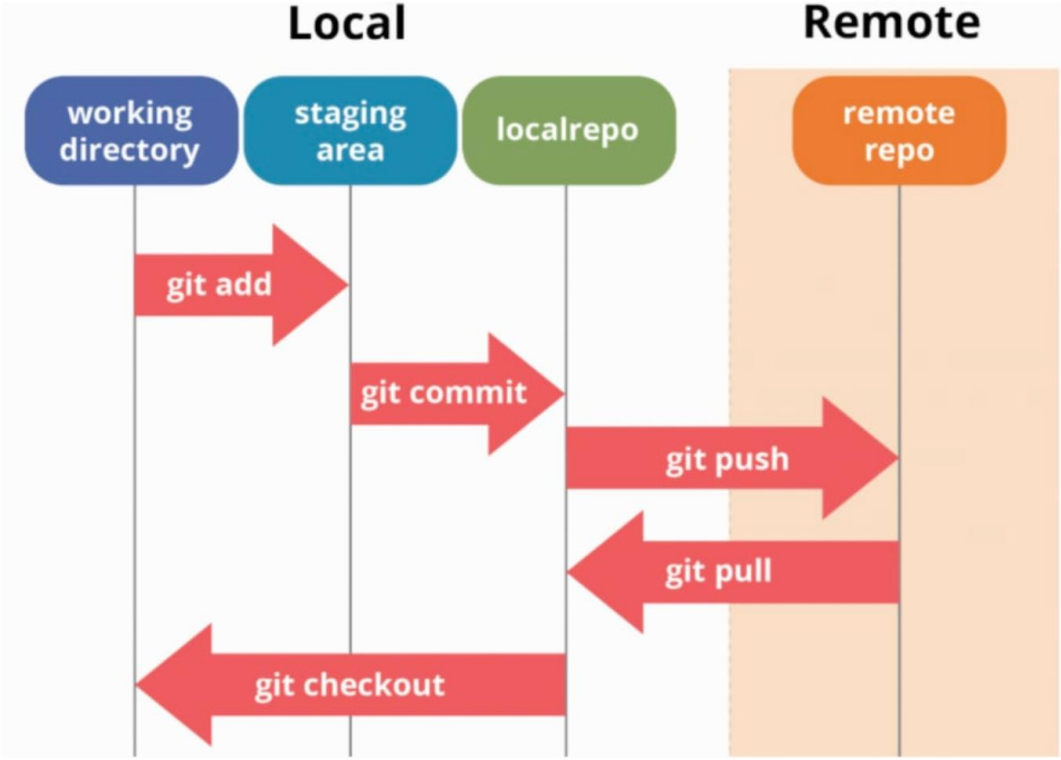
대표적인 시스템:

Git

Git

Git

: 현재 가장 널리 사용되는 분산 버전 관리 시스템



Remote Repository

: 여러 사람과 함께 공유하기 위한 파일을 보관하는 전용 저장소

Local Repository

: 내 PC에 파일이 저장되는 개인 저장소

Index(Staging Area)

: working directory에서 repository로 정보가 저장되기 전 준비 영역

: git 이력과 관련된 정보가 저장되어있는 .git를 제외한 모든 영역

GitHub

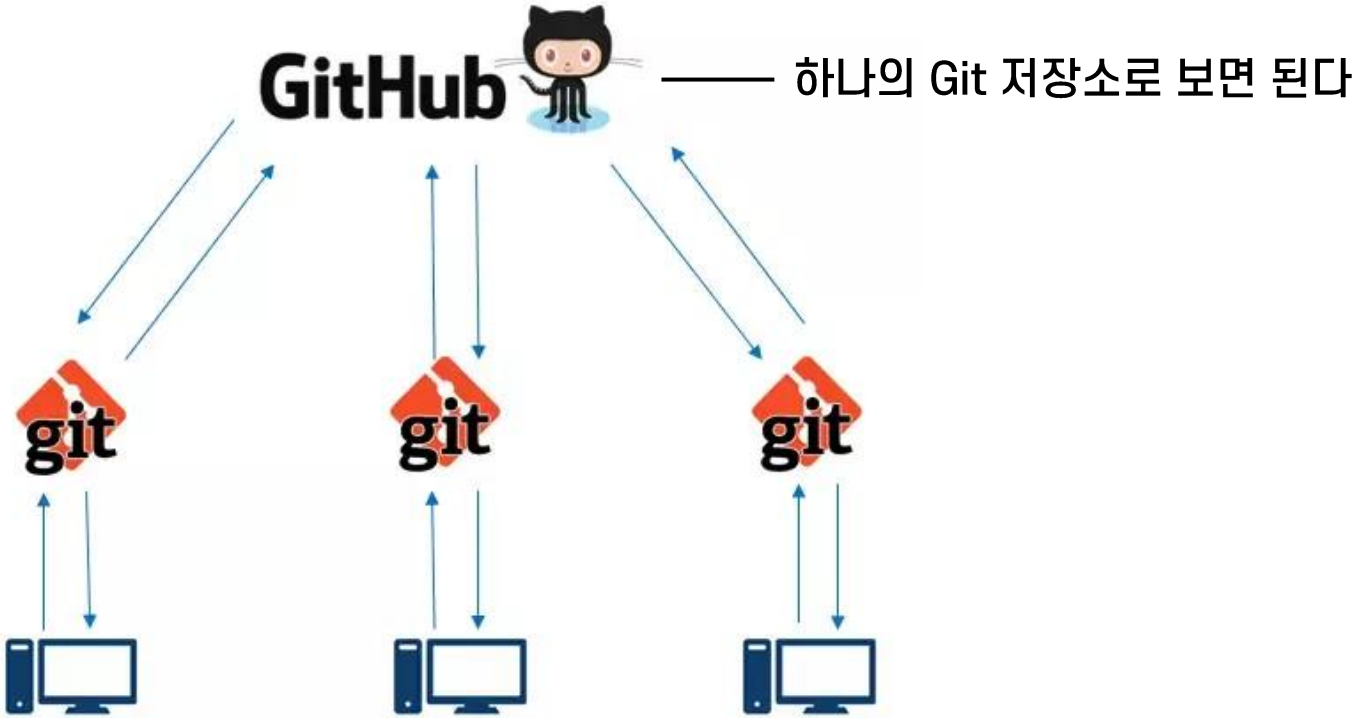
GitHub

: Git 기반의 온라인 저장소 서비스, Git 저장소를 웹에서 관리할 수 있는 플랫폼

개발자가 소스코드를 오픈소스로 공개하겠다는 전제 하에 무료로 저장공간과 도구를 제공

GitHub의 기능

- 코드 저장소(repository) 제공
- 협업 기능
- Issue 관리
- 버그 추적
- 프로젝트 관리
- 게시판 제공
- Pull Request



Git 명령어

git init

: 현재 디렉토리를 Git 저장소(Repository)로 초기화하는 명령어

.git 폴더가 생성되면서 해당 폴더가 Git 관리 대상이 된다.

```
git init
```

git clone

: 원격 저장소에 있는 프로젝트를 내 컴퓨터에 복사하는 명령어(내부적으로 git init 한 후 복사)

```
git clone 저장소URL
```

예시: `git clone https://github.com/user/project.git`

git status

: 현재 Git 저장소의 파일 상태를 확인하는 명령어

```
git status
```

Git 명령어

git add

: 파일을 staging area에 추가하는 명령어

특정 파일 추가: git add 파일명

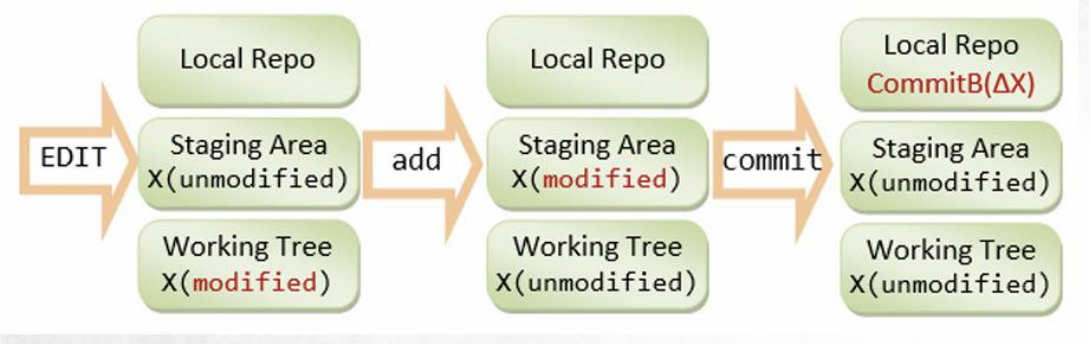
모든 파일 추가: git add .

git commit

: staging area에 있는 변경 사항을 버전으로 저장하는 명령어

git commit -m “메시지”

예시: git commit -m “login 기능 추가 "



git push

: 로컬 저장소의 commit을 원격 저장소에 업로드하는 명령어

git push origin 브랜치명

예시: git push origin main

Git 명령어

git pull

: 원격 저장소의 변경사항을 가져와서 자동으로 merge 하는 명령어

git pull origin 브랜치명

git fetch origin

git merge origin/main

git fetch

: 원격 저장소의 변경사항을 가져오기만 하고 merge는 하지 않는 명령어(현재 작업중인 branch에 반영 안됨)

git fetch origin

git merge

: 다른 브랜치에서 작업한 내용을 현재 브랜치에 병합

git merge 브랜치이름

예시: git switch main

git merge feature-login

feature-login 브랜치가 main으로 병합됨

Git 명령어

git checkout

: 브랜치를 변경하거나 특정 버전으로 이동하는 명령어 + 특정 파일, 특정 버전으로 복구 등 다양한 기능

브랜치 이동: git checkout 브랜치명

브랜치 생성 + 이동: git checkout -b 브랜치명

git switch

: 브랜치를 이동하거나 생성하는 명령어

브랜치 이동: git switch 브랜치명

브랜치 생성 + 이동: git switch -c 브랜치명

Git 명령어

git restore

: 작업 중 변경된 파일을 이전 상태로 되돌리는 명령어

Working directory에서 수정한 파일을 마지막 commit 상태로 복구
: git restore 파일명

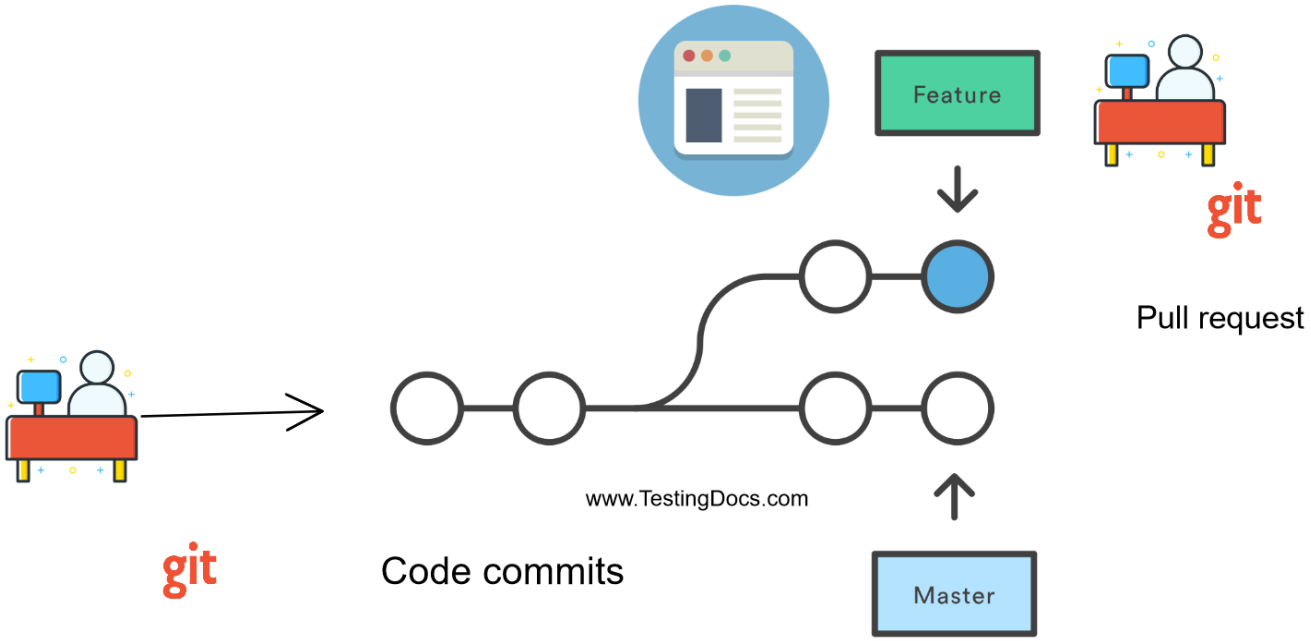
add 취소, staging에 올린 파일을 다시 working directory 상태로 되돌리기
: git restore --staged 파일명

특정 commit 상태로 복구
: git restore --source commitID 파일명

Pull Request

Pull Request

: 개발자가 만든 코드의 변경사항을 프로젝트의 메인 브랜치에 병합해달라고 프로젝트 관리자에게 요청하는 것



1. 개발자가 feature branch 생성
2. 기능 개발
3. add + commit + push
4. Pull Request 생성
5. 코드 리뷰
6. 필요 시 수정
7. request reject 또는 merge